ISIS-1225 · Reto 3: Análisis de ventas de calzado deportivo

Grupo 08 · Análisis de complejidad temporal

Integrantes

Nombre	Código	Correo Uniandes	Responsable de
Nicolas Liz Figueroa	202611971	n.liz@uniandes.edu.co	Req. 1 (individual)
Juan Pablo Martinez	202611361	jp.martinezr1@uniandes.edu.co	Req. 3 (individual)
Alvaro Villate [completar nombre, código y correo]	[completar]	[completar]	Req. 2 (individual)
Grupo completo	—	—	Req. 4, 5 y 6 (grupales)

Notación: **n** = total de ventas del archivo; **m** = ventas de una marca o país (subconjunto indexado); **k** = ventas que cumplen el filtro; **b** = número de marcas (constante, 6 en el dataset). Las tablas de hash (linear probing) hacen get/put en O(1) promedio; el árbol rojo-negro (RBT) hace put/get en O(log m) y es balanceado (altura ≤ 2 log m).

Carga de datos e informe de carga

Se lee el CSV una sola vez. Por cada una de las n filas se convierten los tipos (O(1)), se agrega a la lista de ventas (O(1) amortizado) y se inserta en el RBT de su marca (Req. 1) y en el RBT de su país (Req. 2), cada uno en O(log m).

El informe de carga (menor/mayor precio final, primeros y últimos 5) recorre la lista una vez: O(n).

Complejidad: O(n log n) la carga; O(n) el informe

Req. 1 — Órdenes de una marca con descuento en un rango (Nicolas Liz)

Se obtiene el RBT de la marca en la tabla de hash en O(1). La llave del árbol es la tupla (discount_percent, order_date, order_id), por lo que el recorrido en orden ya entrega descuento ascendente y, en empate, fecha ascendente; el order_id evita llaves repetidas. La consulta de rango (*values*) descarta subárboles fuera del rango: baja O(log m) niveles y visita solo los k nodos que cumplen. Luego un recorrido de la lista resultado acumula el costo total en O(k).

No se requiere ordenar después de filtrar.

Complejidad: O(log m + k) ≤ O(n)

Req. 2 — Órdenes de un país con calificación en un rango (Alvaro Villate)

Misma idea: un RBT por país con llave (customer_rating, order_date, order_id). La consulta de rango cuesta O(log m + k). Para obtener el orden descendente se recorre el resultado insertando al inicio de una lista encadenada (add_first, O(1)), O(k) en total. Los 5 primeros y 5 últimos se extraen en O(k).

Complejidad: O(log m + k)

Req. 3 — Órdenes por canal, categoría y rango de precio final (Juan Pablo Martinez)

Se recorre toda la lista de ventas (O(n)) comparando canal, categoría y precio. Cada una de las k ventas que cumplen se inserta en un RBT con llave (final_price_usd, order_date, order_id): O(k log k). El recorrido en orden del árbol (value_set) entrega el resultado ordenado en O(k).

Complejidad: O(n + k log k)

Req. 4 — Marcas con más órdenes por rango de fechas y nivel de ingresos (grupal)

Un recorrido de la lista (O(n)) filtra por fecha y nivel y acumula, por marca en una tabla de hash, el conteo, las sumas de precio e ingreso y la orden de mayor ingreso (O(1) por venta). Las b marcas se insertan en una cola de prioridad con prioridad (-total, nombre): b inserciones O(b log b). Se extraen las N primeras: O(N log b).

Complejidad: O(n + b log b) = O(n), pues b es constante

Req. 5 — Marcas con mejor calificación promedio en un país y rango de fechas (grupal)

Recorrido único O(n) con filtro por país y fecha; acumula por marca (tabla de hash) conteo, suma de calificaciones, suma de ingresos y la orden con más unidades. Cada marca entra a una cola de prioridad con prioridad (-promedio, nombre) en O(log b) y se extraen las N mejores.

Complejidad: O(n + b log b) = O(n)

Req. 6 — Marcas más consistentes en descuentos (grupal)

Un recorrido O(n) filtra por fecha y precio base y agrupa las ventas por marca (tabla de hash, listas). Luego, para cada marca con n_i ventas, se calcula el promedio, la desviación estándar poblacional y la orden más cercana al promedio en $O(n_i)$; la suma de todas las n_i es $\leq n$. Las marcas se ordenan con una cola de prioridad con prioridad (desviación, promedio, nombre): O(b log b).

Complejidad: O(n + b log b) = O(n)

Observación sobre el Req. 3: a diferencia de los Req. 1 y 2, no se indexa de antemano; por eso su costo depende de n y no de log m. Los Req. 1 y 2 sacrifican O(n log n) en la carga para responder cada consulta en O(log m + k).